

回答以下几个问题：

训练哪些题 + 按什么顺序训练 + 每道题分配多少 rollout + 什么时候淘汰或重放 + 怎样

截至 2026 年 8 月，我建议将相关研究分成五组。

一、与 DRIVE 最直接相似：RLVR 数据筛选与课程学习

1. DRIVE

[DRIVE: Data Curation Best Practices for RLVR in Competitive Code Generation](#) |

[代码](#) DRIVE 的核心流程是：

SFT 阶段

- 按 easy/medium/hard 划分竞赛题；
- 提高难题的采样权重；
- 混合通用代码与高强度推理数据；
- 使用强开源模型蒸馏推理轨迹。

第一阶段 RL

- 大规模、难度分布相对均匀的数据；
- 每题约 8 个 rollout；
- 相对较短的输出窗口；
- 目标是扩大策略熵、减少重复和截断。

第二阶段 Pre-GRPO

- 小规模高质量难题；
- 每题扩大到约 64 个 rollout；
- 持续保留最困难问题；
- hard-focus curriculum。DRIVE 最重要的观点是：

对竞赛代码 RL，题目选择、难度分布和 rollout budget 可能比 GRPO loss 的细微修改更重要。

2. DEPO：离线筛选 + 在线动态过滤 + 重放

Towards High Data Efficiency in Reinforcement Learning with Verifiable Reward

这是与 DRIVE 最接近的通用 RLVR 数据效率工作之一。DEPO 分成离线和在线两部分。

离线筛选

根据三个维度选择训练子集：

- Diversity：题目是否覆盖不同知识和推理模式；
- Influence：该题的梯度对目标能力是否有帮助；
- Appropriate difficulty：是否位于当前模型可学习的难度区间。

在线筛选

定义 sample-level explorability，过滤：

- rollout 全部成功的饱和题；
- rollout 全部失败且暂时没有探索潜力的题；
- 重复采样也几乎无法产生新信号的题。

动态重放

被判断为 under-explored 的题不会永久删除，而是放入 replay buffer，在模型变强后重新尝试。论文报告仅使用约 20% 数据，便可达到或超过全量 GRPO，并在 AIME24/AIME25 上获得约 $1.85\times/1.66\times$ 的训练加速。与 DRIVE 的区别：

DRIVE	DEPO
人工设计两阶段课程	离线与在线自动筛选
强调困难题和大 rollout	强调可探索性与重放
主要针对竞赛代码	主要验证于数学推理

3. CROPI: 用 influence function 选择最有价值的 RL 数据

[Data-Efficient RLVR via Off-Policy Influence Guidance](#) | [代码](#) CROPI 不再仅用当前正确率衡量数据价值，而是问：

在这道题上做一次梯度更新，是否会改善验证集或其他题目的性能？对于训练样本 z_i ，可将其影响近似为：

$$I(z_i) \approx -\nabla_{\theta} L_{\text{val}}^{\top} H_{\theta}^{-1} \nabla_{\theta} L(z_i).$$

直接在线估计非常昂贵，因此 CROPI 使用预收集的离线轨迹近似每个 prompt 的影响，然后只在高价值子集上执行真正的 RL rollout。它比 DRIVE 的“困难题优先”更精细，因为：

- 困难不等于有用；
- 某些困难题梯度噪声很大；
- 某些中等题可能对很多能力都有正迁移；
- influence 更接近“训练这一题是否真的改善目标”。

4. LearnAlign: 从梯度对齐角度选择数据

[LearnAlign: Data Selection for LLM Reinforcement Learning with Verifiable](#)

[Rewards](#) LearnAlign 同样研究 RLVR 数据选择，但重点是梯度表示。它先用模型当前能力估计样本 learnability，然后构造带可学习性权重的梯度表示：

$$g_i^{\text{weighted}} = w_i^{\text{learnability}} \nabla_{\theta} L_i.$$

再计算候选训练数据与目标数据之间的梯度对齐程度：

$$S_i = \cos\left(g_i^{\text{weighted}}, g_{\text{target}}\right).$$

高分样本代表：

- 模型当前能从中学到东西；
- 更新方向与目标能力一致；

- 不只是因为回答长而拥有较大梯度范数。它与 CROPI 属于同一新趋势：从表面难度指标走向 gradient-aware data curation。

5. Learn More with Less: 不确定性一致性选题

Learn More with Less: Uncertainty Consistency Guided Query Selection for RLVR 传统方法常用模型置信度作为难度：

$$u_{\text{subjective}} = \text{entropy}/\text{logit uncertainty}.$$

但模型可能非常自信地答错，也可能不自信地答对。因此主观不确定性未必对应客观正确率。论文同时考虑：

- 主观不确定性：模型 logits 表现出的 uncertainty；
- 客观不确定性：多个 rollout 的实际正确与错误分布；
- uncertainty consistency：二者是否一致。离线版本使用点二列相关系数 PBC；在线版本则结合 normalized advantage 和主观 uncertainty 构造近似指标。实验表明，只使用约 30% 的数据即可达到全数据训练效果。它对 DRIVE 的启发是：竞赛代码不能只按照通过率划分难度，还应该判断模型的置信度是否与单元测试结果一致。例如：



高置信度 + 全错：关键盲区
低置信度 + 偶尔正确：高探索价值
高置信度 + 全对：已饱和
低置信度 + 全错：可能过难或数据有问题

二、动态安排题目训练顺序

6. E2H Reasoner: 从易到难的 RL 课程

Curriculum Reinforcement Learning from Easy to Hard Tasks Improves LLM Reasoning 它是最标准的 easy-to-hard RL 课程：

$$D_1^{\text{easy}} \rightarrow D_2^{\text{medium}} \rightarrow D_3^{\text{hard}}.$$

但论文发现不能一直保留大量简单题。简单题在前期有助于建立基本策略，后期如果仍占很

高比例，则可能：

- 浪费 rollout;
- 使模型过拟合简单模式;
- 阻止能力边界继续外扩。因此需要逐步衰减简单题：

$$p_t(D_{\text{easy}}) \downarrow, \quad p_t(D_{\text{hard}}) \uparrow.$$

它适合小模型，因为小模型直接在困难题上做 RL 时，往往无法采到任何成功轨迹。与 DRIVE 的关系是：

- DRIVE 的第二阶段 hard-focus 与它思想一致;
- E2H 更强调先建立基础能力;
- DRIVE 更强调竞赛题数据分布和 rollout 设置。

7. Re-Schedule: 根据推理树结构而非 pass rate 排课程

[Scheduling Your LLM Reinforcement Learning with Reasoning Trees](#) | [代码](#) 普通课程通常用题目正确率衡量难度：

$$d(x) = 1 - \hat{p}(\text{correct} \mid x).$$

但两道正确率相同的题可能有完全不同的推理空间：

- 题 A：模型总是在两种接近的路径中选择;
- 题 B：模型生成大量完全不同的错误路径;
- B 的结构性探索难度显然更高。Re-Schedule 对每道题构造近似 reasoning tree:
- 每个节点是部分推理前缀;
- 每个分支是不同 continuation;
- 统计成功路径、分支结构和深度;
- 得到结构化难度 r-score。然后按：

高 r-score $\rightarrow$ 低 r-score

进行由结构简单到复杂的课程学习。论文报告相较准确率课程和普通 GRPO，平均准确率最高提升约 3.2%。对 Code RL 来说，reasoning tree 可以替换为：

- 算法设计分支;

- 代码生成分支;
- 编译错误类型;
- 测试失败模式;
- 修复路径。这可能比单纯的 pass rate 更能描述一道代码题的真实学习难度。

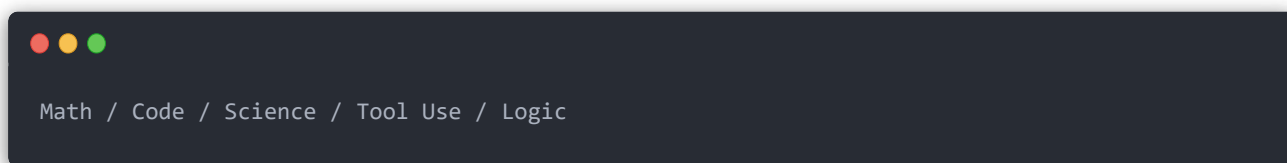
8. TAC: 选择能够帮助其他领域的训练域

An Automated Curriculum for Multi-Domain RLVR Transfer-Aware Curriculum

(TAC) 用于多领域 RLVR。它不仅考察某个领域当前是否可学习，还考察在该领域更新后是否能改善其他领域：

$$S_k = \underbrace{L_k}_{\text{learnability}} + \beta \underbrace{T_k}_{\text{cross-domain transfer}} .$$

其中领域被视为 bandit arm，例如：



如果代码题上的梯度同时改善数学和工具使用，TAC 会提高代码域的采样概率；如果某个领域只改善自己，甚至伤害其他域，权重会降低。相比 DRIVE 的单一竞赛代码课程，TAC 更适合训练通用推理模型。

三、rollout budget 分配：不再每题固定采样 8 或 64 次

DRIVE 使用明确的阶段配置：

- 第一阶段约 8 rollouts;
- 困难阶段约 64 rollouts。2026 年最新研究开始让 rollout 数量自动适应每道题。

9. VIP: 根据预估奖励方差分配 rollout

Adaptive Rollout Allocation for Online RLVR 对于二元 reward，若 prompt 成功概率为 p_i ，其奖励方差为：

$$\text{Var}(R_i) = p_i(1 - p_i).$$

当 $p_i \approx 0.5$ 时方差最大，通常能形成最强组内比较信号；当 $p_i \approx 0$ 或 1 时，组内容易全错或全对。VIP 使用预测模型估计每道题的成功概率与不确定性，再求解预算分配问题：

$$\min_{\{G_i\}} \text{Var}(\hat{g}), \quad \sum_i G_i = B.$$

这比 DRIVE 固定 8/64 更节省计算，但依赖成功率预测质量。

10. Hit-Utility: rollout 的目标是至少命中一次成功

[Hit-Utility Optimal Rollout Allocation for Group-Based RLVR](#) 对于困难 prompt，重要的可能不是最大化奖励方差，而是至少生成一个正确回答：

$$P(\text{hit}) = 1 - (1 - p_i)^{G_i}.$$

当 p_i 很低时，固定 8 个 rollout 可能一个成功也没有；适当提高 G_i 才能跨过零优势区。但对已经很容易的题：

$$p_i \rightarrow 1,$$

继续增加 rollout 几乎没有收益。它直接把 rollout 分配建模成有限预算下的 hit utility 优化，特别接近 DRIVE 的困难题大采样思想，但更加理论化、连续化。

11. VIGOR: 逐轮增加 rollout，只给有方差的题追加预算

Learning as Reasoning Unfolds: Progressive Rollout Allocation for Efficient RL
VIGOR 是截至 2026 年 7 月非常新的工作。它不预先决定每题生成多少次，而是逐步采样：

```

先为每题生成少量 rollout
→ 检查组内奖励分布
→ 全对/全错且判断无价值：停止
→ 有正确有错误：继续分配 rollout
→ 更新方差估计

```

核心依据是组内 reward variance。只有持续产生有效相对优势的题才获得更多计算。 论文报告：

- 数学任务最多减少约 $2.3 \times$ rollout;
 - 代码任务达到 GRPO 最终通过率时减少约 $1.49 \times$ rollout;
 - 代码测试平均通过率同时提升约 3.4 个点。 这可以视为 DRIVE 8→64 两阶段预算的在线自适应版本。
-

12. SARA: 边采样边决定何时停止

Sequential Adaptive Rollout Allocation (SARA) 将 rollout 收集建模为 sequential optimal stopping。 每生成一个新答案，就更新对该 prompt 的判断：

$$P(\text{继续采样有额外价值} \mid r_{1:k}).$$

如果前几个 rollout 已经显示：

- 基本必然全对；
 - 基本必然全错；
 - 现有组已足够形成稳定优势； 就提前停止，把预算转给其他 prompt。 它与 VIGOR 的区别是：
 - VIGOR 偏分阶段逐步追加；
 - SARA 更接近逐样本的 sequential decision；
 - 两者都比固定组大小更节省 rollout。
-

四、持续产生新的可验证训练数据

13. X-Coder: 完全合成竞赛题、解答与测试

[X-Coder: Advancing Competitive Programming with Fully Synthetic Tasks, Solutions, and Tests](#) X-Coder 是与 DRIVE 在代码数据构造方面最相关的新工作之一。它提出 SynthSmith，完整合成：

Problem + Solution + Test cases.

流程不是简单改写已有题，而是从算法与题目特征组合出发生成新题，例如：

- 算法类型；
 - 数据结构；
 - 约束规模；
 - 输入输出格式；
 - 边界情况；
 - 所需复杂度；
 - 干扰性条件。之后执行并验证参考解和测试，使其可用于 SFT 和 RLVR。X-Coder 7B 在 LiveCodeBench v5/v6 上报告 62.9 avg@8 和 55.8，超过一些更大的代码模型。与 DRIVE 的关系：
 - DRIVE 研究已有竞赛数据怎样筛选和安排；
 - X-Coder 研究怎样扩展新的、可验证的竞赛训练数据；
 - 二者组合才是更完整的 Code RL 数据管线。
-

14. Golden Goose: 从不可验证文本生成可验证 RL 题

Golden Goose 很多互联网文本包含高质量知识，但没有明确 verifier，无法直接用于 RLVR。Golden Goose 将原始文本转换为 fill-in-the-middle 式多选题：

1. 找出关键推理片段；
2. 隐去该片段；
3. 生成多个合理但错误的干扰项；
4. 原始片段作为可验证答案。从而把：

unverifiable text $\rightarrow$ verifiable multiple-choice RL task.

作者构建 GooseReason-0.7M，覆盖数学、编程、科学等领域，并在原 RL 数据已经饱和后继续带来收益。它不专门面向代码生成，但对扩大 RLVR 数据规模非常有价值。

15. SVS: 让当前模型生成下一阶段的训练问题

Self-play with Variational Problem Synthesis (SVS) 关注训练数据饱和问题。基本循环是：

$\pi_t \rightarrow$ 发现当前能力边界 $\rightarrow$ 合成边界附近的新题 $\rightarrow RL \rightarrow \pi_{t+1}$.

合成题不能：

- 太像现有题；
- 太简单；
- 完全不可解；
- 无法验证。因此重点是对已有问题做有控制的变体，例如改变条件、组合知识点或增加推理深度。这代表比 DRIVE 的固定训练集更进一步的方向：课程本身随策略共同演化。

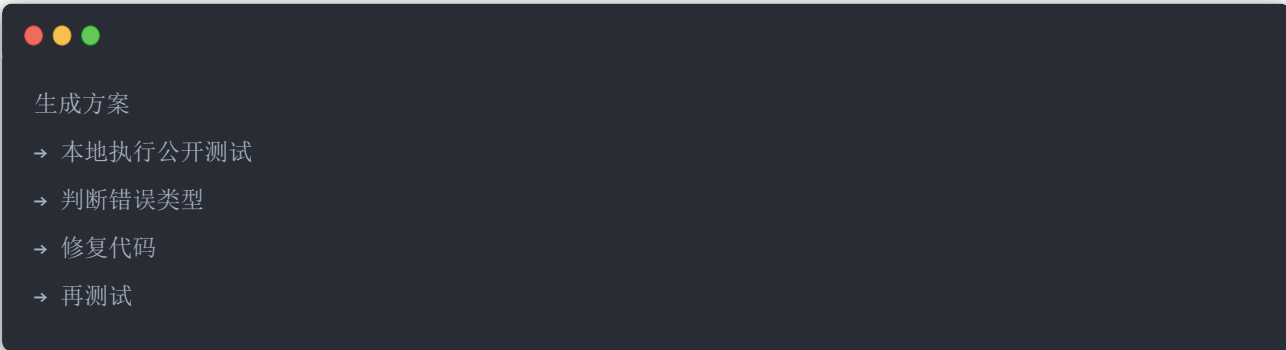
五、代码方向的相邻前沿

以下论文不完全属于“数据策展”，但对构建 DRIVE 之后的 Code RL 管线很重要。

16. RefineRL: 把单次代码生成数据转成多轮修复环境

RefineRL: Advancing Competitive Programming with Self-Refinement RL

RefineRL 让模型反复：



```
生成方案
→ 本地执行公开测试
→ 判断错误类型
→ 修复代码
→ 再测试
```

Skeptical-Agent 即使公共测试已经通过，也不会立即认为答案绝对正确，而是继续检查潜在边界错误，防止对稀疏公共测试过拟合。它说明竞赛代码 RL 的“数据单位”正在从：

(problem, one-shot solution)

变成：

(problem, multi-turn refinement trajectory).

这会影响未来的数据筛选标准：不仅选择题目，还要选择哪些失败轨迹最值得修复。

六、推荐优先级

如果你的目标是延续 DRIVE，研究 “Code RLVR 数据工程” ，我建议重点读下面 10 篇：

优先级	论文	主要价值
1	DRIVE	竞赛代码数据课程基线
2	DEPO	离线筛选、在线过滤和动态重放
3	VIGOR	动态 rollout budget，包含代码实验
4	X-Coder	全合成竞赛题、解答和测试
5	CROPI	influence-guided 数据选择
6	LearnAlign	梯度对齐式数据选择
7	Re-Schedule	reasoning-tree 难度课程
8	Learn More with Less	不确定性一致性选题
9	E2H Reasoner	静态 easy-to-hard 基础范式
10	RefineRL	从单轮题目扩展到多轮代码修复

七、这些工作正在形成的统一框架

可以把最新 RLVR 数据管线概括成：



然后将失败题重新处理：

all correct → 淘汰或提高难度，

mixed outcomes → 优先训练，

all wrong but promising → 追加 rollout、重放或提供提示，

all wrong and unlearnable → 分解、合成简单变体或暂缓。

DRIVE 主要完成了 “人工分阶段课程” 的实践验证；DEPO、CROPI、LearnAlign、VIGOR 等新工作正在把每一个阶段自动化。如果面向竞赛代码，我认为最有潜力的组合是：

这比单纯继续修改 GRPO advantage 更可能产生一条完整且有研究价值的 Code RL 数据训练路线。